

CyberCrypto

Abeer Alsafran

Cryptography

System Design

Conclusion

Algorithms

Introduction

Attacks

Background

Result

Introduction

CyberCrypto is a secure end-to-end encryption system used to secure the communication between Server and Client - aka - Alice and Bob.

Goal

Goal

1. Security reasoning and a formal understanding of cryptographic principles.
2. Cryptographic design choices and their trade-offs.
3. Awareness of potential attacks and resistance strategies.
4. Performance analysis and scalability considerations.
5. Connections to current research and real-world applications.
6. Simulation of a secure end-to-end communication protocol for two or more parties under realistic threat models.

Background

Symmetric

Asymmetric

Symmetric

- AES (Advanced Encryption Standard):

A widely adopted symmetric encryption standard endorsed by NIST for national and industrial use. It is available in 128-bit, 192-bit, and 256-bit key sizes and offers high performance and security.

- DES (Data Encryption Standard):

Formerly popular, DES is now considered obsolete due to its vulnerability to brute-force attacks. It uses a 56-bit key size and has been replaced by more secure alternatives like AES and 3DES.

- Triple DES (3DES):

An improved version of DES, which applies the DES algorithm three times to each data block. Although stronger than DES, it is slower and less efficient than AES.

Asymmetric

- RSA (Rivest-Shamir-Adleman): Based on the difficulty of factoring large prime numbers; widely used in HTTPS, VPNs, and secure email.
- ECC (Elliptic Curve Cryptography): Utilizes elliptic curves; provides strong security with shorter keys, making it ideal for IoT and mobile applications.
- DSA (Digital Signature Algorithm): Specializes in creating digital signatures.

Algorithms

For CyperCrypto we have used the following algorithms:

1- Deffie-Hellman

2- RSA

3- HMAC

4- AES

DH

RSA

HMAC

AES

DH

one prime P and G (a primitive root of P) and two private values a and b .

Alice	Bob
Public Keys available = P, G	Public Keys available = P, G
Private Key Selected = a	Private Key Selected = b
Key generated = $x = G^a \bmod P$	Key generated = $y = G^b \bmod P$
Exchange of generated keys takes place	
Key received = y	key received = x
Generated Secret Key = $k_a = y^a \bmod P$	Generated Secret Key = $k_b = x^b \bmod P$
Algebraically, it can be shown that $k_a = k_b$	
Users now have a symmetric secret key to encrypt	

Alice

```
# -----
# Handshake by Diffie Hellman
# -----

parameters = dh.generate_parameters(generator=2, key_size=2048)
server_private_key = parameters.generate_private_key()
server_public_key = server_private_key.public_key()
parameters_numbers = parameters.parameter_numbers()
p = parameters_numbers.p
g = parameters_numbers.g

# \\ Step 1 - send our public key \\
server_public_bytes = server_public_key.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo,
)

proto.send_server_params(conn, p, g, server_public_bytes)
```

Bob

```
pn = dh.DHParameterNumbers(msg["p"], msg["g"])
client_parameters = pn.parameters()
print("Generating DH key pair for client.")
client_private_key = client_parameters.generate_private_key()
client_public_key = client_private_key.public_key()
print("Key pair ready.\n")

client_public_bytes = client_public_key.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo,
)

client_shared_key = client_private_key.exchange(loaded_server_public_key)
session_key = cu.generate_dh_keypair(client_shared_key)
```



Prezi

Algorithms

For OpenCrypto we have used the following algorithms

RSA

1. Key Generation

- Choose two large prime numbers, say p and q . These prime numbers should be kept secret.
- Calculate the product of primes, $n = p * q$. This product is part of the public as well as the private key.
- Calculate Euler Totient Function $\Phi(n)$ as $\Phi(n) = \Phi(p * q) = \Phi(p) * \Phi(q) = (p - 1) * (q - 1)$.
- Choose encryption exponent e , such that
 - $1 < e < \Phi(n)$, and
 - $\gcd(e, \Phi(n)) = 1$, that is e should be co-prime with $\Phi(n)$.
- Calculate decryption exponent d , such that
 - $(d * e) \equiv 1 \pmod{\Phi(n)}$, that is d is modular multiplicative inverse of $e \pmod{\Phi(n)}$. Some common methods to calculate multiplicative inverse are: Extended Euclidean Algorithm, Fermat's Little Theorem, etc.
 - We can have multiple values of d satisfying $(d * e) \equiv 1 \pmod{\Phi(n)}$ but it does not matter which value we choose as all of them are valid keys and will result into same message on decryption.

Finally, the **Public Key** = (n, e) and the **Private Key** = (n, d) .

Bob - client

```
# ----- #
# RSA KEY FOR SIG                               #
# ----- #

# \ Send to the server the client public key so that he can verify the signature \
print("Generating RSA-2048 key pair for client..")
rsa_client_private_key, rsa_client_public_key = cu.generate_rsa_keypair() # create keys
rsa_client_public_key_pem = cu.serialize_public_key(rsa_client_public_key) # serialize them
# \ send to server rsa public key for verification \
proto.send_client_params(sock, rsa_client_public_key_pem, rsa=True)
```

Alice - server

```
# ----- #
# Receive Client RSA pub for verification        #
# ----- #

msg = proto.recv_dh(conn)
if msg["type"] != proto.T_HELLO:
    proto.send_error(conn, "Expected SALAM")
    return

rsa_client_public_key = serialization.load_pem_public_key(msg["rsa_client_public_key_pem"])
```

HMAC

The formula for HMAC:

$$\text{HMAC} = \text{hashFunc}(\text{secret key} + \text{message})$$

```
def generate_hmac(key: bytes, msg: bytes) -> bytes:
    """Generates a HMAC signature for a message."""
    return hmac.new(key, msg, hashlib.sha256).digest()

def verify_hmac(key: bytes, msg: bytes, received_signature: bytes) -> bool:
    """Verifies a message's integrity against a received signature."""
    expected_signature = generate_hmac(key, msg)
    return hmac.compare_digest(expected_signature, received_signature)
```

Bob - client

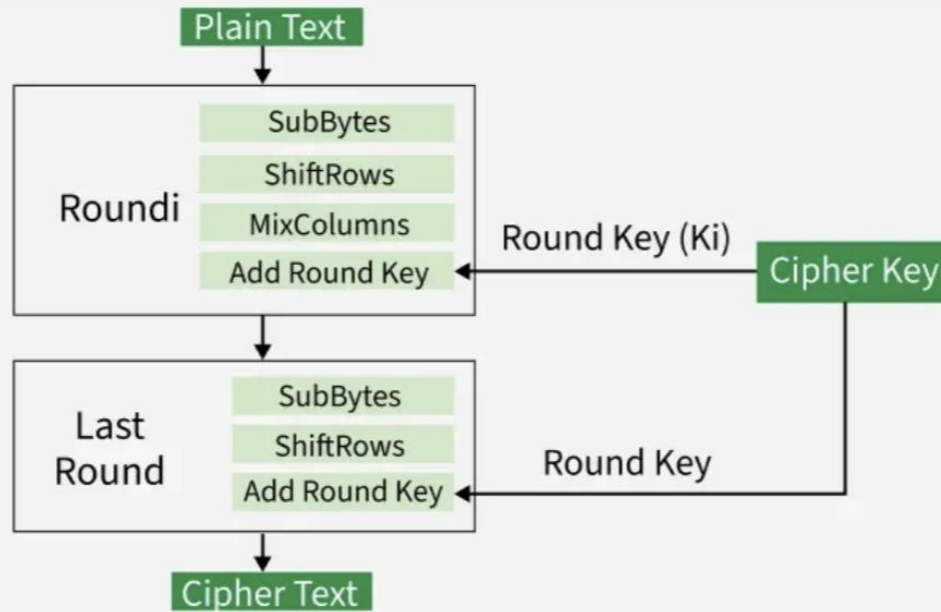
```
if scenario["use_hmac"]:
    hmac_msg = cu.generate_hmac(session_key, plaintext_bytes)
```

Alice - server

```
hmac_auth = cu.verify_hmac(session_key, plaintext, hmac_msg)
if not hmac_auth:
    print("Message is not verified : Probability of message tampering!\n")
    print(f"[{peer}] Verification time: {time.perf_counter() - message_start:.6f} seconds")
    continue
```

AES

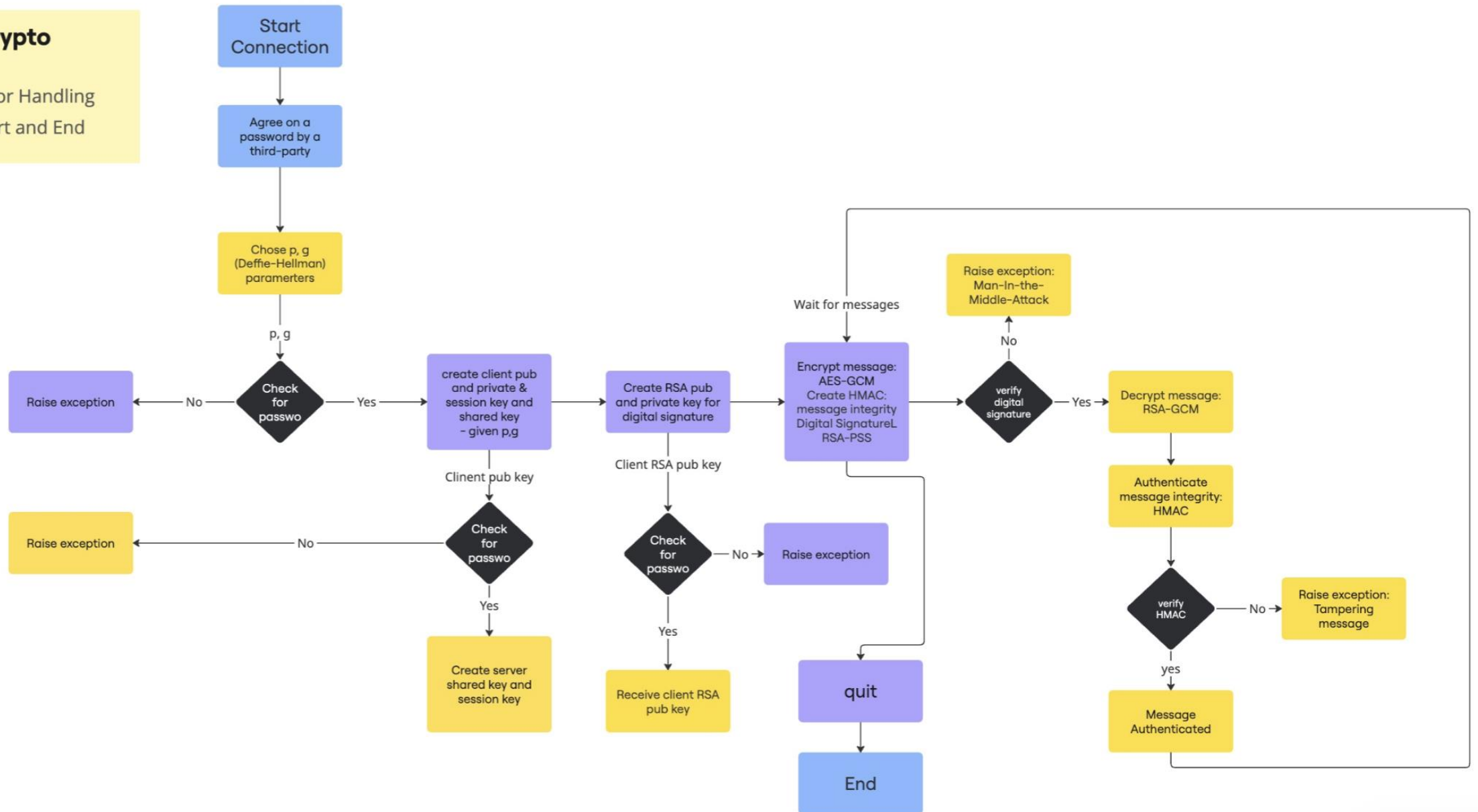
AES-GCM: Advanced Encryption Standard Galois/Counter Mode



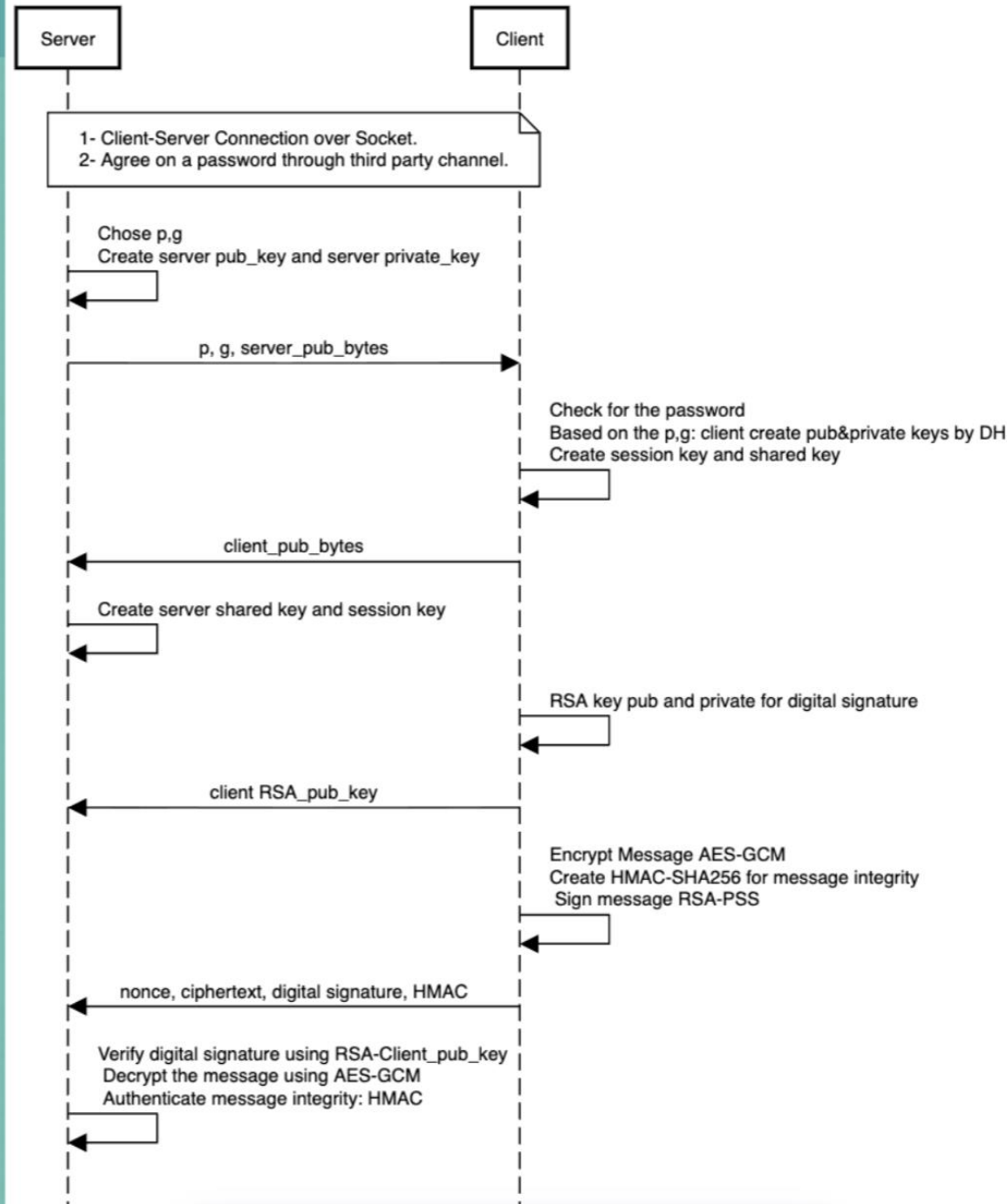
```
def aes_encrypt(key: bytes, plaintext: bytes) -> tuple[bytes, bytes]:  
    """  
    Encrypt *plaintext* with AES-256-GCM.  
  
    Returns (nonce, ciphertext+tag). The tag is appended to the ciphertext  
    by the AESGCM implementation automatically.  
    """  
    nonce = os.urandom(NONCE_SIZE)  
    aesgcm = AESGCM(key)  
    ciphertext = aesgcm.encrypt(nonce, plaintext, None)  
    return nonce, ciphertext  
  
def aes_decrypt(key: bytes, nonce: bytes, ciphertext: bytes) -> bytes:  
    """  
    Decrypt *ciphertext* (with appended GCM tag) using *key* and *nonce*.  
  
    Raises cryptography.exceptions.InvalidTag if authentication fails.  
    """  
    aesgcm = AESGCM(key)  
    return aesgcm.decrypt(nonce, ciphertext, None)
```

CyberCrypto

Server ■ Error Handling
Client ■ Start and End



CyberCrypto Sequence Diffie Hellman Key Exchange



Attack Simulations

1. No Diffie-Hellman key exchange (using random prime numbers) | Attack: Brute force.
2. HMAC verification | Attack: Message tampering.
3. Digital Signature (DS) verification | Attack: Man-in-the-Middle (MITM).
4. No Digital Signature (DS) and no HMAC.

Result

No.	Test Case	Handshake Time	Attack Time	Verification Time
1	No DH	5.97	0.00896	0.00107
2	HMAC Verification	9.80	0.00744	0.000597
3	DS Verification	<u>5.93</u>	0.00789	0.000397
4	No DS, No HMAC	25.56	<u>0.00356</u>	<u>0.000151</u>

Table 2: CyberCrypto Scheme | All time in second | Averaged on 4 attempts

No.	Test Case	Handshake Time	Attack Time	Verification Time
1	CyberCrypto Scheme	6.56	0.00795	0.000660

Conclusion

Our security analysis showed that the proposed design can effectively detect and mitigate several realistic threats, including, message tampering and man-in-the-middle attacks.

Limitations

Demo

Key Limitations

One of the most challenging issue we have faced is that the model was only tested on a small amount of input length, which does not cover all edge cases.

Another issue is the variability of the handshake time; sometimes, the same run for the handshake takes longer than usual to establish a session key, but some times it takes less time.

Resolution

For the small number of input, we tried to maximize the message size limit.

As for the time variation, we tried to test the simulated attacks in a stable environment, multiple time and take the average.

Demo

Thank you!